main.py

Visualize

Run ▶

```python
def string_matching(words):
    result = []

    for word in words:
        for other in words:
            if word != other and word in other:
                result.append(word)
                break

    return result


words = ["mass", "as", "hero", "superhero"]
print(string_matching(words))
```

Output

```
['as', 'hero']
```

main.py

Visualize

Run ▶

```python

def closest_pair(points):
    min_distance = float('inf')
    closest = None

    for i in range(len(points)):
        for j in range(i + 1, len(points)):
            d = distance(points[i], points[j])

            if d < min_distance:
                min_distance = d
                closest = (points[i], points[j])

    return closest, min_distance


points = [(1, 2), (4, 5), (7, 8), (3, 1)]

pair, min_dist = closest_pair(points)

print("Closest pair:", pair[0], "-", pair[1])
print("Minimum distance:", min_dist)
```

Output

```
Closest pair: (1, 2) - (3, 1)
Minimum distance: 2.23606797749979
```

main.py

Visualize | Run ▶

```
                cross = cross_product(
                    points[i], points[j], points[k]
                )

                if cross > 0:
                    left = True
                elif cross < 0:
                    right = True

            if not (left and right):
                hull.add(points[i])
                hull.add(points[j])

    return list(hull)


points = [
    (10, 0), (11, 5), (5, 3), (9, 3.5),
    (15, 3), (12.5, 7), (6, 6.5), (7.5, 4.5)
]

print("Convex Hull:", convex_hull(points))
```

Output

```
Convex Hull: [(6, 6.5), (10, 0), (15, 3), (5, 3), (12.5, 7)]
```

main.py

Visualize

Run ▶

```
            if not (left and right):
                hull.add(points[i])
                hull.add(points[j])

    hull = list(hull)

    if len(hull) <= 2:
        return hull

    cx = sum(p[0] for p in hull) / len(hull)
    cy = sum(p[1] for p in hull) / len(hull)

    hull.sort(
        key=lambda p: math.atan2(p[1] - cy, p[0] - cx)
    )

    return hull


points = [(1, 1), (4, 6), (8, 1), (0, 0), (3, 3)]

print("Convex Hull:", convex_hull(points))
```

Output

```
Convex Hull: [(0, 0), (8, 1), (4, 6)]
```

main.py

Visualize | Run ▶

```
    for perm in permutations(remaining):
        path = [start] + list(perm) + [start]
        total = 0

        for i in range(len(path) - 1):
            total += distance(path[i], path[i + 1])

        if total < min_distance:
            min_distance = total
            best_path = path

    return min_distance, best_path


cities1 = [(1, 2), (4, 5), (7, 1), (3, 6)]
cities2 = [(2, 4), (8, 1), (1, 7), (6, 3), (5, 9)]

for i, cities in enumerate([cities1, cities2], start=1):
    d, path = tsp(cities)
    print("Test Case", i)
    print("Shortest Distance:", d)
    print("Shortest Path:", path)
```

Output

```
Test Case 1
Shortest Distance: 16.969112047670894
Shortest Path: [(1, 2), (7, 1), (4, 5), (3, 6), (1, 2)]
Test Case 2
Shortest Distance: 23.12995011084934
Shortest Path: [(2, 4), (6, 3), (8, 1), (5, 9), (1, 7), (2, 4)]
```

main.py

Visualize

Run ▶

```
    return best_assignment, min_cost


cost1 = [
    [3, 10, 7],
    [8, 5, 12],
    [4, 6, 9]
]

cost2 = [
    [15, 9, 4],
    [8, 7, 18],
    [6, 12, 11]
]

for i, matrix in enumerate([cost1, cost2], start=1):
    assignment, cost = assignment_problem(matrix)

    print("Test Case", i)
    print("Optimal Assignment:",
          [(f"Worker {w+1}", f"Task {t+1}")
           for w, t in enumerate(assignment)])
```

Output

```
Test Case 1
Optimal Assignment: [('Worker 1', 'Task 3'), ('Worker 2', 'Task 2'), ('Worker 3',
'Task 1')]
Total Cost: 16
Test Case 2
Optimal Assignment: [('Worker 1', 'Task 3'), ('Worker 2', 'Task 2'), ('Worker 3',
'Task 1')]
Total Cost: 17
```

main.py

Visualize

Run ▶

```
                    max_value = value
                    best_items = list(items)

    return best_items, max_value


weights1 = [2, 3, 1]
values1 = [4, 5, 3]
capacity1 = 4

weights2 = [1, 2, 3, 4]
values2 = [2, 4, 6, 3]
capacity2 = 6

for i, (weights, values, capacity) in enumerate([
    (weights1, values1, capacity1),
    (weights2, values2, capacity2)
], start=1):
    items, value = knapsack(weights, values, capacity)

    print("Test Case", i)
    print("Optimal Selection:", items)
    print("Total Value:", value)
```

Output

```
Test Case 1
Optimal Selection: [1, 2]
Total Value: 8
Test Case 2
Optimal Selection: [0, 1, 2]
Total Value: 12
```